Project Report

Learn and Give Back to Society

Predicting Credit Card Risk with Machine Learning Classification Models

Logistic Regression, Random Forest and XGBoost Machine Learning Models

Romin Gandhi & Jenish Bharucha

Gandhi, Romin; Bharucha, Jenish

Table of contents

List of Figures

List of Tables

Preface

This report was completed for the final project in CP 322: Machine Learning at Wilfrid Laurier University by Romin Gandhi and Jenish Bharucha.

The reason why credit risk was chosen as a topic is that, being two students with a background in both computer science and finance, we wanted to work on a problem that involves both fields, and credit risk is an important issue in financial services organizations. According to data, credit card charge-offs in Q3 2024 have risen to 4.65%, the highest level since 2011 (National Creditors Bar Association 2024). It shows that there should be a better way to predict whether or not an applicant will default on their credit card balance. With this project, we hope to show our understanding of various machine learning models that will help predict whether or not an applicant will default on their credit card balance, along with the trade-offs between each of the three models used to solve the problem.

Abstract

The purpose of this project is to help financial organizations deal with the challenges they face in evaluating credit risk by applying classification models on the potential credit card applicants who may default in paying the credit card. The issue with financial organizations is that the credit scoring model is based on linear methods and has a certain number of predefined variables. However, with the help of a machine learning model, the difference is that it would be helpful in analyzing complex data and dealing with non-linear data

The dataset used for this project is from Kaggle called Credit Card Approval Prediction dataset which consists of two CSV files: application_record.csv with 438,000+ applicant records across 18 features, and credit_record.csv with monthly payment history for each applicant. The final dataset after preprocessing included 36,457 applicants with a default rate of just 1.69% after merging and creating a classification default target variable, which means that the dataset has a class imbalance.

The models were trained on train/test split ratio of 80 and 20 respectively and class imbalance was handled using SMOTE oversampling for Logistic Regression and balanced class weighting for Random Forest. For XGBoost, the class imbalance problem was handled using `scale_pos_weight`. All hyperparameters for the models were chosen using `GridSearchCV` and `RandomizedSearchCV` with 5-fold cross-validation.

While all three models were able to achieve the accuracy of 80%, they were very different in terms of precision-recall tradeoff. The accuracy of the Random Forest model was the highest at 98.48%, with precision at 67.65%. However, the recall of the model was very low at just 18.70%, indicating that the model was missing most of the defaulters. On the other hand, the Logistic Regression model achieved the highest recall at 71.54%, but the precision was very low at just 6.04%. This indicates that the Logistic Regression model had a lot of false positives. The XGBoost model had the best performance with the highest ROC AUC at 88.88% and an F1-score at 47.15%. Therefore, the XGBoost model was the best for predicting credit defaulters.

Chapter 1

Introduction

1.1 Overview

In the present financial service scenario, the process of credit risk assessment is an important part of the overall process. In this project, three different classifiers, namely, Logistic Regression, Random Forest, and XGBoost, are used to predict the default of the credit card based on the data set provided by the Kaggle Credit Card Approval Prediction Data Set, a binary classification problem.

1.1.1 Existing System

In the traditional approach to credit risk assessment, the rule-based method and the logistic regression method are used. The problem with these methods is that biased decision-making is involved, and also these methods are not capable of handling non-linear data. Moreover, these methods are also not scalable.The problem is also that the credit default data is imbalanced, where one class is highly represented, and the other class is less represented, i.e., 1.7%. Naive classifiers have high accuracy because one class is highly represented, and the defaulters are ignored.

1.2 Objectives

The objectives of this project are as follows:

1. **Data preparation**: This objective involves merging 2 raw CSV files into 1 CSV file which combines the applicant demographic data with their monthly credit payment records, and aggregate each applicant's payment history into a single row. This will be the processed CSV.
2. **Preprocessing and feature engineering**: This objective is created so that all missing values are handled before models are trained, outliers are capped using the interquartile range method, categorical features are encoded as numerical features and new features are created such as age in years, employment duration, and income per family member from existing features.

3. **Model development**: The machine learning models will be classification models of increasing complexity starting with logistic regression as a linear baseline, and random forest and XGBoost as more powerful tree based alternatives for this type of large dataset.
4. **Class imbalance handling**: The data set contains a class imbalance problem, and it will be solved using SMOTE oversampling, balanced class weighing, and scaling of the positive class weight to equally represent the minority class during the training of the model.
5. **Hyperparameter optimization**: Each of the three classification models will have hyperparameters selected through grid search and five fold cross validation to find the best performing parameter combination.
6. **Evaluation and comparison**: All three models will be tested on a test dataset which will not be used during the training of the models. Performance of each model will be compared based on 5 metrics which include accuracy, precision, recall, F1-Score, and ROC AUC.

Chapter 2

Pre-Processing and Exploratory Data Analysis

2.1 Dataset Collection

2.1.1 Dataset Description

The data used in this project is sourced from the Credit Card Approval Prediction dataset on Kaggle (Kaggle 2023). This dataset consists of two CSV files:

application_record.csv contains 438,557 records across 18 attributes describing the demographic and repayment behaviour of each of the applicants:

Table 2.1: Attributes in application_record.csv

Attribute	Description
ID	Unique applicant identifier
CODE_GENDER	Gender (M/F)
FLAG_OWN_CAR	Car ownership (Y/N)
FLAG_OWN_REALTY	Property ownership (Y/N)
CNT_CHILDREN	Number of children
AMT_INCOME_TOTAL	Annual income
NAME_INCOME_TYPE	Income category (e.g., Working, Pensioner)
NAME_EDUCATION_TYPE	Education level
NAME_FAMILY_STATUS	Marital status
NAME_HOUSING_TYPE	Housing situation
DAYS_BIRTH	Age in days (negative, relative to application date)
DAYS_EMPLOYED	Employment duration in days (negative)
FLAG_MOBIL	Mobile phone ownership
FLAG_WORK_PHONE	Work phone availability

Attribute	Description
FLAG_PHONE	Home phone availability
FLAG_EMAIL	Email availability
OCCUPATION_TYPE	Occupation category
CNT_FAM_MEMBERS	Number of family members

credit_record.csv contains 1,048,575 monthly payment records across 3 columns:

Table 2.2: Columns in credit_record.csv

Column	Description
ID	Applicant identifier (links to application_record)
MONTHS_BALANCE	Month of the record relative to the observation point (0 = current month, -1 = previous month, etc.)
STATUS	Monthly repayment status code (see below)

Table 2.3: STATUS code definitions

Status Code	Meaning
0	1–29 days past due
1	30–59 days past due
2	60–89 days past due
3	90–119 days past due
4	120–149 days past due
5	150+ days past due or written off
C	Paid off that month
X	No loan that month

2.1.2 Target Variable Definition

The target variable `default` is created based on the entire credit history provided for each applicant from the dataset. If there is any particular month for which STATUS is 2, 3, 4, or 5, then the applicant is deemed as a defaulter (i.e., `default` = 1), and for all other applicants, `default` = 0.

2.1.3 Data Merging and Record Counts

The joining of the application records with the credit records is a many-to-one relationship, as there are multiple months of credit history available for each applicant. To get a single row of data per applicant, which can then be used for classification, the monthly credit records are aggregated by calculating:

- The number of times each **STATUS** value occurs per applicant (how many months were in status C, how many in status 0, etc.)
- The total number of months on record, along with the minimum and maximum months

After merging and creating the target variable, there are 36,457 unique applicants in the dataset. Of these, 35,841 (98.3%) are non-defaulters, and 616 (1.7%) are defaulters, indicating a significant class imbalance problem to be addressed in model training.

2.2 Data Pre-processing

2.2.1 Missing Values

After merging the two raw CSV files, the dataset had two types of missing values:

1. `OCCUPATION_TYPE` Column: 11,323 missing values (31.1%)

This was the only column that had missing values. Since over one-third of applicants did not input their occupation, filling the blank rows of OCCUPATION_TYPE column with an "Unknown" label made the most sense because labelling it with any real occupation would be misleading. This way the "Unknown" label becomes its own category and it was encoded as 17 as a numerical category.

2. `DAYS_EMPLOYED` Column: 6,135 placeholder values (16.8%)

This column presents 2 issues. First it uses a placeholder value of 365,243 for applicants who are unemployed or retired. This means that if left as is, it would represent 1,000 years of employment which would skew the model badly. Instead all placeholder value of 365,243 were replaced with an value of 0 to represent applicants being unemployed or retired. Second, this column then was converted into `years_employed` by taking the absolute value and dividing by 365 days because the column has negative values. The original `DAYS_EMPLOYED` column was then dropped.

2.2.2 Outliers

Outliers in the numerical columns were handled by being capped rather than removed to keep all the rows in the dataset. Column `AMT_INCOME_TOTAL` was a column with outliers because the income of applicants ranged from $27,000 to $1,575,000 with a mean of $186,686 and a standard deviation of $101,789, which created a right skewed distribution. The values in this column were capped at $380,250 from the maximum value of $1,575,000 using IQR-based winsorization.

2.2.3 Scaling

All of the numerical columns were scaled using `StandardScaler` which means that each column was rescaled to have a mean of zero and a variance of 1. This was necessary for all 3 models since they all read from the same preprocessed `train.csv` and `test.csv`. However it is the most important for the Logistic Regression model because it is sensitive to feature magnitude.

2.2.4 Feature Engineering and Transformation

New features were created from the raw data. This includes:

1. **age_years** was created from DAYS_BIRTH by taking the absolute value since it used negative day count and dividing by 365. DAYS_BIRTH column was then dropped.
2. **years_employed** was created from DAYS_EMPLOYED after replacing the 365,243 placeholder with 0. DAYS_EMPLOYED column was then dropped.
3. **income_per_member** was created by taking the AMT_INCOME_TOTAL / CNT_FAM_MEMBERS. The reason was to capture the financial burden per household member.
4. **income_per_child** was created by taking the AMT_INCOME_TOTAL / CNT_CHILDREN for applicants who had children. It was set to 0 otherwise, to help capture the financial burden from dependents.

All categorical columns were then label-encoded. This includes: CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, and OCCUPATION_TYPE columns.

The data went through 2 more filtering steps which were:

1. **Low-variance removal** (threshold < 0.01): those columns that were nearly identical across all applicants were dropped since they provided no useful information.
2. **High-correlation removal** (threshold > 0.9): where if two columns were highly correlated then one was removed to avoid redundancy.

2.3 Exploratory Data Analysis and Visualizations

2.3.1 Data Visualization

Target Variable Distribution

A bar chart was plotted for the target variable "default", which is highly imbalanced. Out of 36,457 applicants, there were 35,841 (98.31%) non defaulters, and only 616 (1.69%) were defaulters. The imbalance ratio was 58.2. What this means is that a model will predict "No Default" for every applicant with an 98.31% accuracy. This means that accuracy is not a good way to measure in this problem. This is an major reason to use SMOTE and class weighting in all three models.

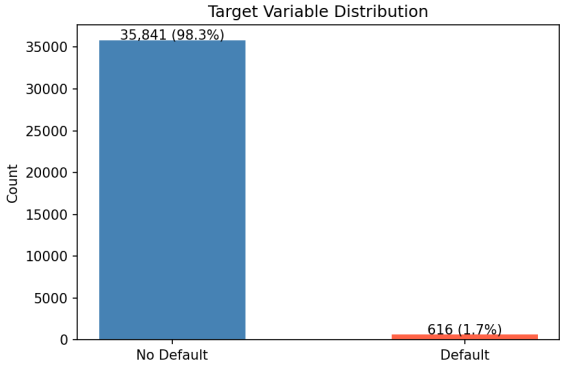


Figure 2.1: Target Variable Distribution

Histograms of Numerical Features

Histograms were created for all the numerical columns. Most columns had right-skewed distributions. For `AMT_INCOME_TOTAL`, the distribution was skewed to the right because there was a small number of high-income applicants. The histogram for `DAYS_EMPLOYED` column had a spike at 365,243, which is due to it being a placeholder.

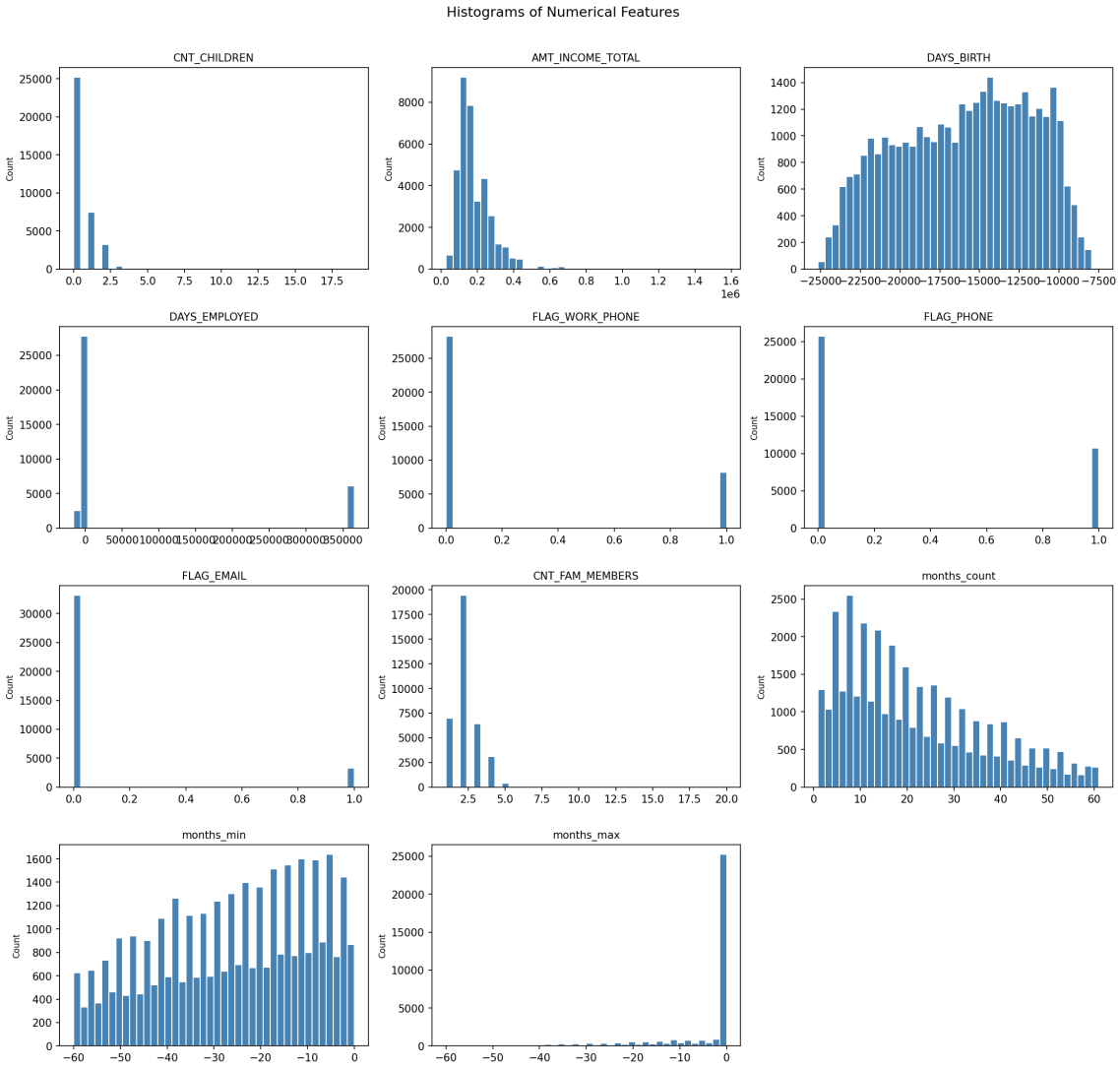


Figure 2.2: Histograms of Numerical Features

Categorical Feature Bar Charts

Bar charts were created for all 8 categorical features. The dominant groups were female applicants at 67%, no car ownership at 62%, property ownership at 67%, working income type at 52%, secondary education at 68%, married at 69%, house/apartment housing at 89% and for those applicants that provided their occupation laborers were the highest at 25.7%.

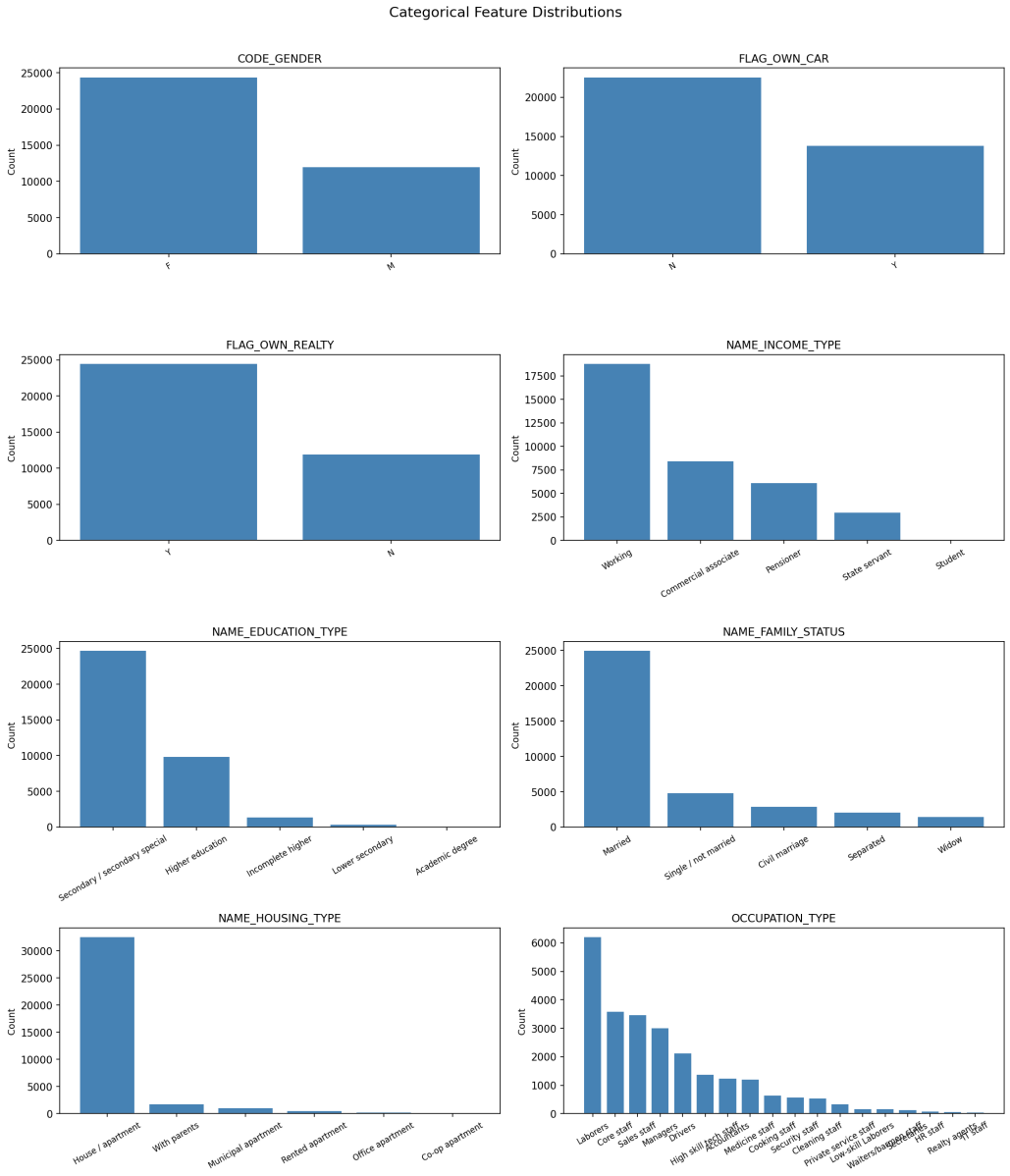


Figure 2.3: Categorical Feature Distributions

Box Plots by Default Status

Box plots were used to compare the distribution of each numerical feature for defaulters and non-defaulters. Most features had similar medians and distributions for both groups. The only feature that had different medians for the two groups was `months_count`, in which the median for defaulters was 35 months compared to 18 months for non-defaulters. This makes sense because defaulters had longer credit history. However, none of the features could be used to separate the two classes cleanly, which confirms that the model will need to use a set of features instead of just a single feature.

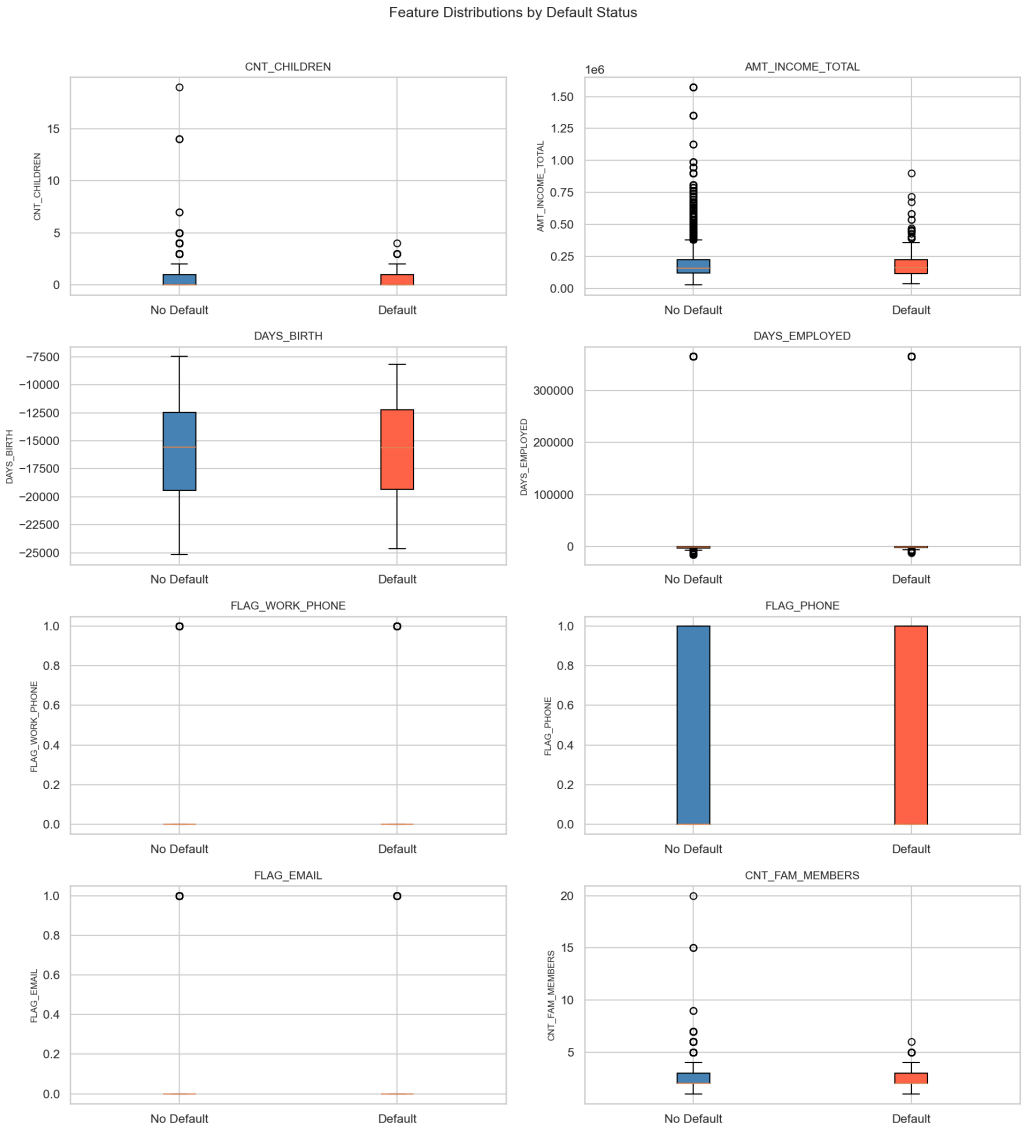


Figure 2.4: Feature Distributions by Default Status

2.3.2 Statistical Analysis

Statistical analysis was performed on the data to identify any patterns, trends, and relationships. What was found during the statistical analysis was that:

1. `AMT_INCOME_TOTAL` ranged from $27,000 to $1,575,000 with a mean of $186,686 and a standard deviation of $101,789.
2. `months_count` ranged from 1 to 61 months, indicating that applicants had very different lengths of credit history.
3. The credit status features from `status_0` to `status_X` indicated that most people in this dataset had little to no history of missed payments, which aligns with the 1.69% default rate.

A correlation heatmap was created on the demographic and financial numerical features with the `default` target variable, as shown in Figure 2.5. The highest correlation is with `months_count` at 0.106, while all other features were below 0.07, indicating they have a weak linear relationship with `default`.

For features, `CNT_CHILDREN` and `CNT_FAM_MEMBERS` are found to have the highest correlation at 0.889, which is below the high correlation value of 0.9, so neither of these features are removed.

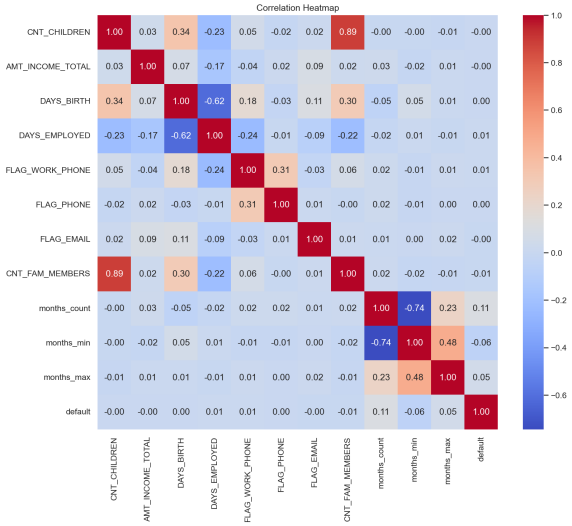


Figure 2.5: Correlation Heatmap of Demographic and Financial Features

2.3.3 Feature Selection

Features that will be used in the model were selected through 2 ways:

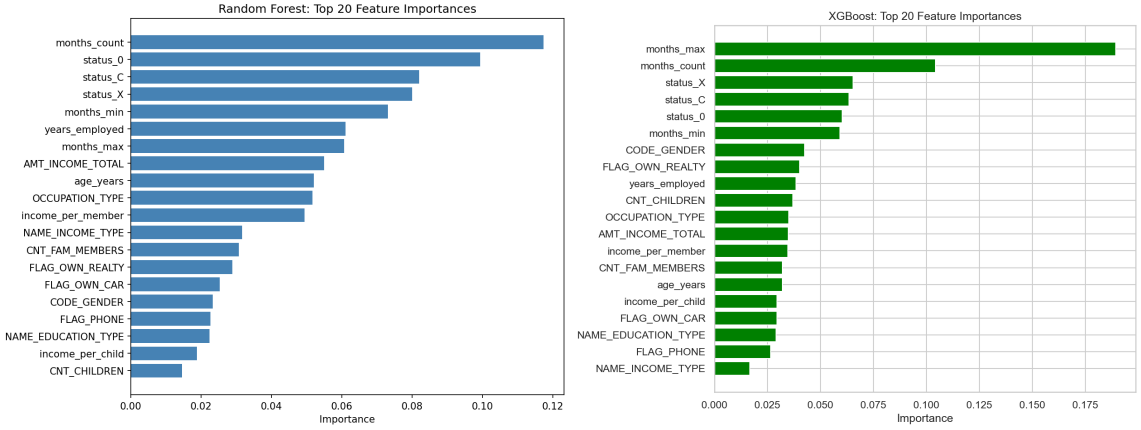
Absolute Correlation with Target

Features were selected based on the correlation between the feature and the target variable default, and taking the absolute value of that correlation. The most important features were all credit history based, such as `months_count` which had a correlation of 0.106, `months_min` which had a

correlation of 0.060, and `months_max`which had a correlation of 0.052. The income level of the applicant, number of children, and age were all below 0.002. This indicates that these variables are not as important on their own but are much more important when combined together.

Mutual Information

The second approach was used to score features based on how well they predict default. The top features were once again related to credit history such as monthly payment status count and credit history length. The demographic features were significantly lower in both, Random Forest and XGBoost model, which means that the payment history is by far the best predictor of the target variable default. The feature importance plots show this, in which `months_count`, `status_0`, and `status_C` were ranked as the top features in both models.



(a) Random Forest Top 20 Feature

(b) XGBoost Top 20 Feature

Figure 2.6: Top 20 Feature Importances

2.3.4 Dimensionality Reduction

No dimensionality reduction techniques were used in this project. The dataset was already reduced to 28 features after filtering, dimensionality of the data did not become a problem in any of the three models. Just removing features that were low variance and high correlation were enough to clean up the features.

Chapter 3

Methodology

3.1 Introduction to Python for Machine Learning

Python was the programming language that was used for this project because of all the machine learning libraries that exist for building and evaluating models. The following libraries were used and can be found in the requirements.txt as well:

- **pandas** and **numpy** for loading, merging, and manipulating the datasets
- **scikit-learn** for preprocessing, model training, hyperparameter tuning, and evaluation of the models
- **imbalanced-learn** for SMOTE oversampling to handle class imbalance
- **xgboost** for the XGBoost classifier
- **joblib** for saving and loading trained models
- **matplotlib** and **seaborn** for all Visualizations

3.2 Platform and Machine Configurations Used

The project was created and tested on local machine running Windows 11 and MAC OS using Python 3.13 as the programming language. There was no cloud services used in this project except for Github for collaboration. The three machine learning models were saved on the disk using joblib. This allowed to test the models once they were trained faster since they were already saved on disk and did not require rerunning them.

3.3 Data Split

After preprocessing the data, the dataset contained 36,457 applicants which was split into 2 sets: a training set and a test set using an 80/20 split ratio. The data was split using `train_test_split` from scikit-learn with `stratify=y` and `random_state=42`. The reason to use this was to make sure both sets had the same default rate of 1.69%. The test dataset is not introduced at any stage except for evaluating and comparing model performance.

Table 3.1: Train/test split summary

Split	Rows	Default Rate
Training set	29,165	1.69%
Test set	7,292	1.69%

3.4 Model Planning

The first step in picking the model was to determine what type of problem this dataset presented in terms of whether it was a classification problem or regression problem. The data set presented a classification problem since the model needed to determine whether or not the applicant defaults.

The three classification models that were selected to predict whether an applicant defaults or not were Logistic Regression, Random Forest, and XGBoost. Other classification models were discussed but not chosen because of constraints. For example, Support Vector Machine (SVM) would have performed poorly due to the dataset being significantly large, the training time would be significant and with class imbalance, Support Vector Machine does not have any built in mechanism. K-Nearest Neighbors (KNN) was another model that was considered, but because of the curse of dimensionality, it was ruled out. K-Nearest Neighbors relies on euclidean distance to find similar applicants so as the features increase the distance between all the points become similar and meaningless. In short, the dataset requires a tree based model that would be able to handle class imbalance and a large dataset.

3.5 Model Training

All three models were trained only on the training set as mentioned above. The class imbalance of 58.2 is handled differently for each model:

1. Random Forest model uses SMOTE to address the problem of class imbalance by creating more defaulter data points during training because the data set only contains 1.69% of actual defaulters. The class_weight='balanced' parameter ensures that the model pays more attention to defaulters.

2. Logistic Regression uses the SMOTE as well, with class_weight tuned as a hyperparameter during cross-validation.

3. XGBoost model handles class imbalances by using `scale_pos_weight`, which is set to the ratio of non-defaulters to defaulters of 58.2. This means that the defaulters' class is given 58.2 times the weight of the non-defaulter class in the training process.

3.6 Model Evaluation

Each of the three models is tested on the 20% data set which is separated from the training data set. The remaining data set is used to train the models and select the hyperparameters which is done using cross-validation. The test data set is introduced at the very end to test the model on unseen data.

3.7 Model Optimization

Hyperparameters for all three models were tuned using cross-validation.

Random Forest used `RandomizedSearchCV` with 5-fold cross-validation scored on F1.

Table 3.2: Random Forest hyperparameter search space

Hyperparameter	Values searched
`n_estimators`	100, 200, 300, 500
`max_depth`	10, 20, 30, None
`min_samples_split`	2, 5, 10
`min_samples_leaf`	1, 2, 4
`max_features`	sqrt, log2

Logistic Regression used `GridSearchCV` with 5-fold cross-validation scored on accuracy.

Table 3.3: Logistic Regression hyperparameter search space

Hyperparameter	Values searched
`C`	0.001, 0.01, 0.1, 1, 10, 100
`penalty`	l1, l2
`solver`	liblinear, saga
`class_weight`	balanced, None

XGBoost used `RandomizedSearchCV` with 5-fold cross-validation scored on F1.

Table 3.4: XGBoost hyperparameter search space

Hyperparameter	Values searched
`n_estimators`	100, 200, 300, 500
`max_depth`	3, 5, 7, 10
`learning_rate`	0.01, 0.05, 0.1, 0.2
`subsample`	0.6, 0.8, 1.0
`colsample_bytree`	0.6, 0.8, 1.0

3.8 Final Model Building

After hyperparameter tuning, the best parameters found for each model were:

Random Forest (`RandomizedSearchCV`):

Table 3.5: Random Forest best hyperparameters

Hyperparameter	Best Value
`n_estimators`	200
`max_depth`	30
`min_samples_split`	2
`min_samples_leaf`	1
`max_features`	log2

Logistic Regression (`GridSearchCV`):

Table 3.6: Logistic Regression best hyperparameters

Hyperparameter	Best Value
`C`	1
`penalty`	l1
`solver`	saga
`class_weight`	balanced

XGBoost (`RandomizedSearchCV`):

Table 3.7: XGBoost best hyperparameters

Hyperparameter	Best Value
`n_estimators`	300
`max_depth`	5
`learning_rate`	0.2
`subsample`	1.0
`colsample_bytree`	0.8

Once the hyperparameters were chosen, each model was then retrained on the training dataset with these parameters. SMOTE oversampling was used before fitting the Random Forest and Logistic Regression model. In terms of the imbalanced data for XGBoost, `scale_pos_weight=58.2` was used. The three models were then saved to disk using joblib for evaluation on the test dataset.

Chapter 4

Results

4.1 Description of the Models

This problem of credit risk assessment requires classification models. The three classification models that were used are:

1. Logistic Regression: is used as it is one of the simplest model to compare with. It looks at an applicant's features and estimates the likelihood of applicants defaulting. However, it assumes a linear relationship between the features and target variable.

2. Random Forest: is an ensemble of decision trees where each of the tree is trained on a random subset of the data and features. The prediction from the tree is made by majority vote. This model tends to have better performance than logistic regression because it's able to capture complex patterns.

3. XGBoost: is a type of gradient boosting algorithm where each tree is built by correcting the mistakes of the previous tree. This model is a strong performer on structured data compared to Random Forest as it fixes the mistake that the previous tree made.

4.2 Performance Metrics

Each model was evaluated using five metrics:

- **Accuracy**: is the percentage (%) of total correct predictions
- **Precision**: means out of all applicants the model flagged as defaulters, how many actually were defaulters
- **Recall**: means out of all actual defaulters, how many did the model actually catch
- **F1-Score**: is a score that balances both precision and recall
- **ROC AUC**: measures the model's ability to separate defaulters from non-defaulters

However, since the data is heavily imbalanced, the best evaluation metric for this model is the ROC AUC Score given that a model which can accurately predict "No Default" every time would have an

accuracy of 98.31%.

4.3 Results Table

The table below summarizes the performance of all three models on a test set of 7,292 applicants.

Table 4.1: Model performance on the test set (n = 7,292)

Model	Accuracy	Precision	Recall	F1-Score	ROC AUC
Logistic Regression	0.8075	0.0604	0.7154	0.1114	0.8058
Random Forest	0.9848	0.6765	0.1870	0.2930	0.8830
XGBoost	**0.9822**	**0.4715**	**0.4715**	**0.4715**	**0.8888**

The XGBoost model was able to achieve the highest ROC AUC of 0.8888 and is the best model.

4.4 Interpretation of the Results

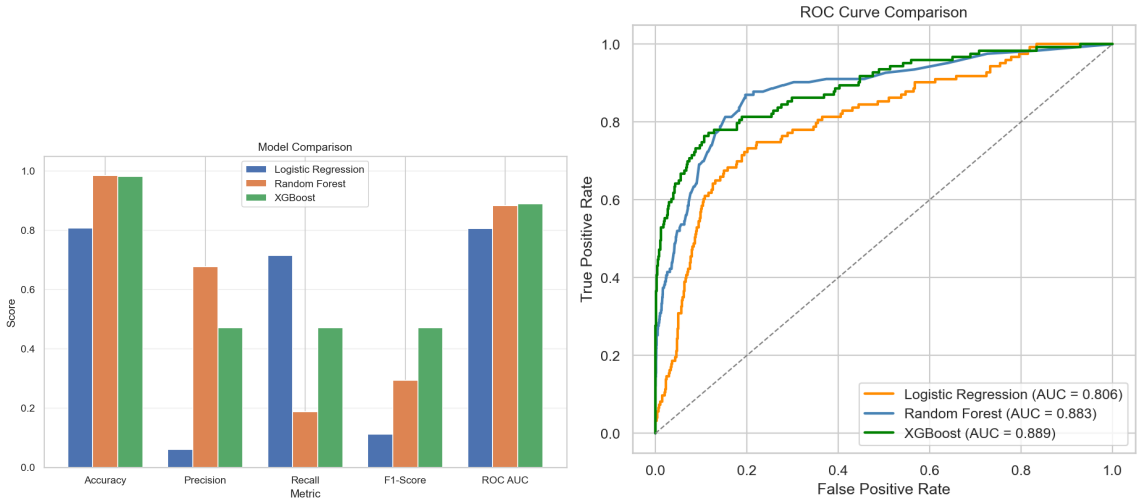
Logistic Regression had the worst ROC AUC at 80.58% and a low precision of 6.04%. The model was able to predict 71.54% of actual defaulters but it also had a false positive rate of 1,369 out of 7,292. This means that the models underlying assumption of the relationship between features and target variable was wrong, it is not linear.

The Random Forest Model had the highest accuracy and precision at 98.48% and 67.65% respectively which means that when the model did predict an applicant as a defaulter it was correct of the times. However the downside to this model is that the recall is 18.70% which means it only caught 18.70% of the actual defaulters. Overall, the model was able to correctly classify 7,158 non-defaulters but only predicted 23 defaulters correctly and missed 100 defaulters.

The recall of the XGBoost model is 47.15%, which means that the model correctly classified 58 of the 123 defaulters at a false positive rate of 65. This means that it is a much better precision-recall tradeoff compared to the other two models. Also, the recall of the Random Forest model is much less compared to the current model. The F1 Score and ROC AUC for the current model is 47.15% and 88.88% respectively, which is the highest among the three models.

4.5 Visualization

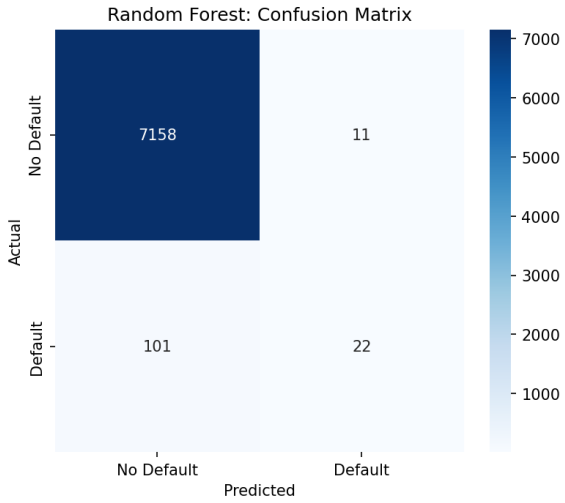
The ROC curves show that all three models perform better than randomly guessing, but it's clear that XGBoost and Random Forest clearly outperform Logistic Regression due to the assumption of a linear relationship. The bar chart of the metrics across the 3 models makes it clear that Logistic Regression model has high recall but an low precision, Random Forest has high precision but an low recall, and XGBoost sits in between with the best balance.



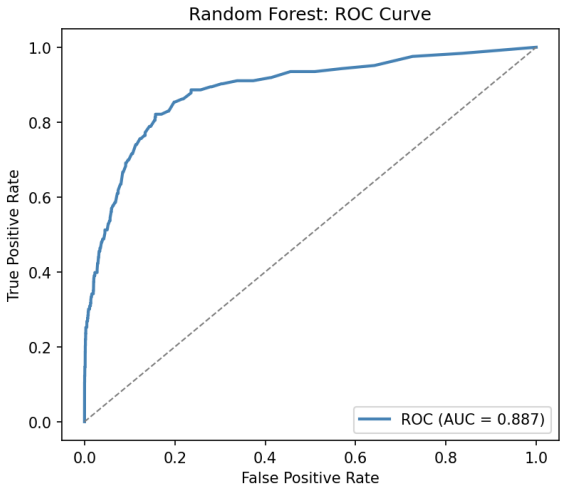
(a) Model Comparison — All Metrics

(b) ROC Curve Comparison

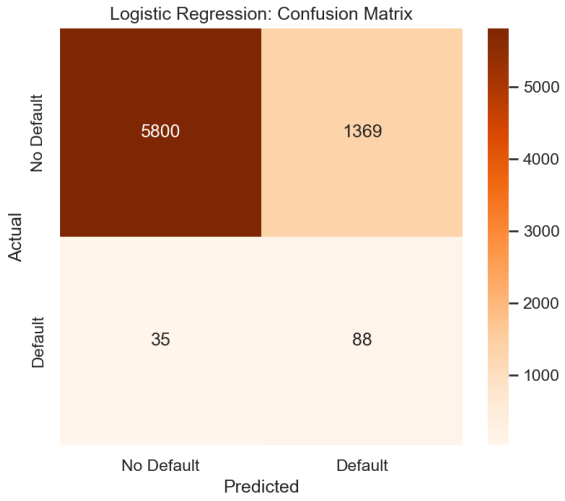
Figure 4.1: Model Performance Visualizations



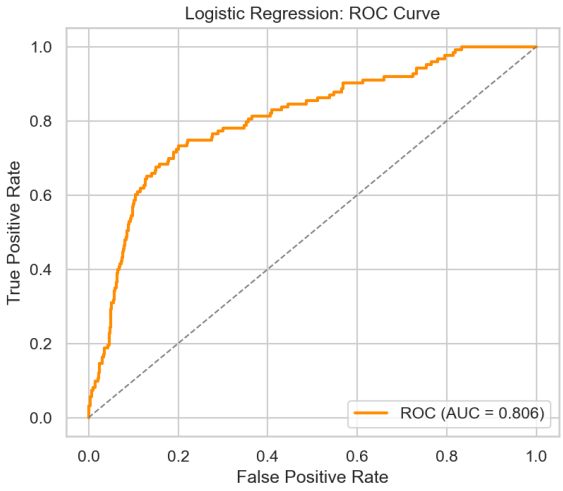
(a) Random Forest Confusion Matrix



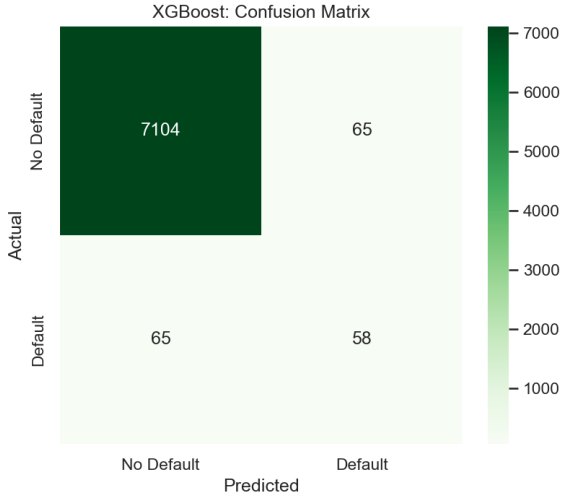
(b) Random Forest ROC Curve



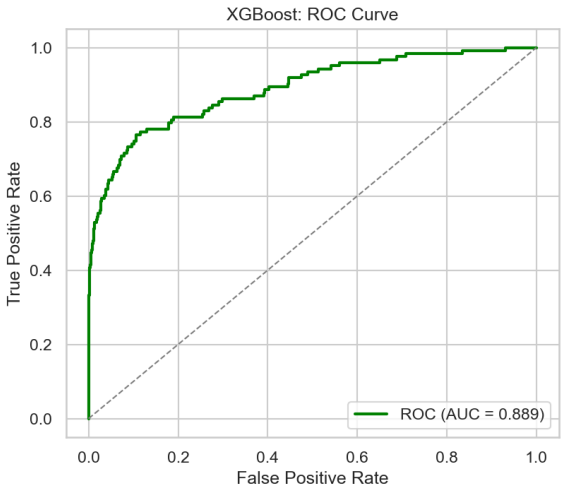
(c) Logistic Regression Confusion Matrix



(d) Logistic Regression ROC Curve



(e) XGBoost Confusion Matrix



(f) XGBoost ROC Curve

Figure 4.2: Individual Model Results — Confusion Matrices and ROC Curves

4.6 Sensitivity Analysis

5-fold cross-validation was run on the training set for each model during hyperparameter tuning. Below are the results of the 5-fold cross-validation:

Table 4.2: 5-fold cross-validation results on the training set

Model	CV Accuracy	CV ROC AUC
Logistic Regression	0.8053 (+/- 0.0069)	0.8191 (+/- 0.0128)
Random Forest	0.9857 (+/- 0.0011)	0.8857 (+/- 0.0102)
XGBoost	0.9837 (+/- 0.0015)	0.9114 (+/- 0.0064)

All three of the models showed low variance across folds, which means that the models are stable. Random Forest and XGBoost both had very small standard deviations of +/- 0.001, which means that their results are consistent and not highly dependent on what rows were included in each fold.

Chapter 5

Conclusion

5.1 Summary of Findings

This project has designed and compared three different machine learning classifier algorithms in predicting credit card defaulters: Logistic Regression, Random Forest, and XGBoost. These classifier algorithms were tested on 36,457 applicant records obtained from the Kaggle dataset on credit card approval prediction, based on a binary target variable on whether an applicant has ever been 60 or more days past due on credit obligations.

All three classification models were able to achieve an accuracy level above 80% on the test dataset. However, there were significant differences in their performance:

- **Random Forest** achieved the best test accuracy of around 98.5%, with high precision and moderate recall on the credit card defaulters class. It used SMOTE oversampling along with balanced class weight and used randomized search on tree depth, number of estimators, and split criteria.
- **XGBoost** resulted in comparable accuracy of around 98.3% with the best ROC AUC across all the models. Hence, it resulted in the best model to predict defaulters and non-defaulters. The use of scale_pos_weight in handling unbalanced data, along with the capacity of gradient boosting to focus more on misclassified data, gave a more balanced precision-recall trade-off than the Random Forest model.
- **Logistic Regression** resulted in an accuracy of around 80.8%, which is the best for the default class in terms of recall but significantly poor for precision. This is because of the trade-off between a linear model and high rebalancing of the classes. The model is good at predicting actual defaulters but also predicts a large number of non-defaulters as defaulters.

Based on the ROC AUC metric, XGBoost was identified as the best performing model for this particular problem of predicting credit defaulters. This is because of the model's ability to balance out the minority class of defaulters with a good classification accuracy, making it the best model for use in a credit risk assessment scenario.

5.2 Future Work

In the future, the results can be improved with better approaches. These include:

1. **Deep learning approaches:** Sequential monthly credit data could be used to train Recurrent Neural Networks (RNN) or transformer models, which can capture the payment patterns lost during aggregation of the data.
2. **Advanced imbalance techniques:** To handle imbalance data more effectively, other methods such as ADASYN, Borderline-SMOTE, or cost-sensitive learning could improve recall but at the cost of precision.
3. **Model ensembling:** The predictions made by Logistic Regression, Random Forest, and XGBoost models can be combined to make one single model, which can be more accurate in its predictions
4. **External data:** More data can be used to improve model accuracy, such as transactional spending data and economic indicators.
5. **Fairness and bias auditing:** The predictions of the models can be tested across different groups of applicants to ensure there is no bias in the predictions, which can be an important point to consider during decision-making.

References

Kaggle. 2023. *Credit Card Approval Prediction*. https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction.

National Creditors Bar Association. 2024. *Credit Card Charge-Offs and Delinquencies Hit 13-Year High: Are They Peaking?* https://www.creditorsbar.org/news/-credit-card-charge-offs-and-delinquencies-hit-13-year-high-are-they-peaking.